CPROG Rapport för Programmeringsprojektet

[Gruppmedlemmar: Robin Plog Jansson 020220

1. Beskrivning

Programmeringsprojektet består av två delar:

1. En generell spelmotor (Game Engine) skriven i C++.
2. Ett spel som använder spelmotorns programmeringsgränssnitt.

Spelmotorn ansvarar för:

- Spelloop (game loop)
- Hantering av sprites
- Rendering av grafik
- Input från tangentbord och mus
- Kollisionsdetektering
- Hantering av objekt som skapas och tas bort

Spelmotorn är designad så att nya spel kan byggas genom att skapa nya klasser som ärver från basklassen Sprite.

Det specifika spelet som implementerats är ett **2D top-down shooter spel**.

I spelet styr spelaren en figur som kan röra sig över skärmen och skjuta projektiler mot fiender. Fiender spawnar högst upp på skärmen och rör sig nedåt. Om en fiende når botten av skärmen avslutas spelet och texten **GAME OVER** visas.

2. Instruktion för att bygga och testa

Programmet är skrivet i **C++** och använder följande bibliotek:

- SDL3
- SDL3_image
- SDL3_ttf

Projektet har utvecklats och testats i **Visual Studio Code** med **MinGW**.

För att bygga projektet krävs att SDL3, SDL3_image och SDL3_ttf är installerade.

Exempel på kompilering:

g++ main.cpp src/engine/*.cpp src/game/*.cpp
-Iinclude -Iinclude/engine -Iinclude/game
-lSDL3 -lSDL3_image -lSDL3_ttf
-o game

Efter kompilering kan programmet startas med:

./game

Resursfiler ligger i katalogen:

resources/

Den innehåller:

images/
fonts/

Bilder som används i spelet:

Player.png
Enemy.png
Bullet.png

Typsnitt:

PressStart2P-Regular.ttf

För att testa spelet:

1. Starta programmet.
2. Flytta spelaren med W A S D.
3. Skjut projektiler med SPACE.
4. Kontrollera att projektiler kan förstöra fiender.
5. Kontrollera att spelet visar **GAME OVER** när en fiende når botten av skärmen.

3. Krav på den Generella Delen(Spelmotorn)

3.1. [Ja/Nej/Delvis] Programmet kodas i C++ och grafikbiblioteket SDL används.
Kommentar: Ja

Programmet är skrivet i C++ och använder grafikbiblioteket SDL. SDL3 används för fönsterhantering, rendering och input. SDL3_image används för att ladda bilder och SDL3_ttf används för text rendering.

3.2. [Ja/Nej/Delvis] Objektorienterad programmering används, dvs. programmet är uppdelat i klasser och använder av oo-tekniker som inkapsling, arv och polymorfism.
Kommentar: Ja

Programmet använder objektorienterad programmering. Programmet är uppdelat i flera klasser såsom:

GameEngine
Sprite
Renderer
World

Spelets objekt (Player, Enemy, Bullet) är implementerade som klasser som ärver från basklassen Sprite. Polymorfism används genom virtuella funktioner såsom tick, onEvent och render.

3.3. [Ja/Nej/Delvis] Tillämpningsprogrammeraren skyddas mot att använda värdesemantik för objekt av polymorfa klasser.
Kommentar: Ja

Värdesemantik för polymorfa objekt förhindras genom att kopieringskonstruktorn och kopieringsoperatorn i Sprite-klassen är borttagna (delete). Objekt hanteras istället via pekare (std::unique_ptr<Sprite>).

3.4. [Ja/Nej/Delvis] Det finns en gemensam basklass för alla figurer(rörliga objekt), och denna basklass är förberedd för att vara en rotklass i en klasshierarki.
Kommentar: Ja

Det finns en gemensam basklass Sprite som representerar alla visuella objekt i spelet. Denna klass fungerar som rot i klasshierarkin och kan ärvas av nya objekt i spelet.

3.5. [Ja/Nej/Delvis] Inkapsling: datamedlemmar är privata, om inte ange skäl.
Kommentar: Ja

Datamedlemmar i klasser är deklarerade som private för att uppnå inkapsling. Åtkomst sker genom publika medlemsfunktioner.

3.6. [Ja/Nej/Delvis] Det finns inte något minnesläckage, dvs. jag har testat och försökt se till att dynamiskt allokerat minne städas bort.
Kommentar: Ja

Dynamiskt allokerade objekt hanteras med std::unique_ptr. Detta säkerställer att minnet automatiskt frigörs när objekten tas bort. SDL-resurser såsom textures och fönster frigörs i destruktorn.

3.7. [Ja/Nej/Delvis] Spelmotorn kan ta emot input (tangentbordshändelser, mushändelser) och reagera på dem enligt tillämpningsprogrammets önskemål, eller vidarebefordra dem till tillämpningens objekt.
Kommentar: Ja

Spelmotorn hanterar input från tangentbord och mus via SDL eventsystem. Dessa händelser översätts till interna InputEvent-objekt och skickas vidare till spelets Sprite-objekt via funktionen onEvent.

3.8. [Ja/Nej/Delvis] Spelmotorn har stöd för kollisionsdetektering: dvs. det går att kolla om en Sprite har kolliderat med en annan Sprite.
Kommentar: Ja

Spelmotorn stödjer kollisionsdetektering genom rektangelbaserad kollisionskontroll. Funktionen queryIntersections i klassen World används för att kontrollera om två sprites kolliderar.

3.9. [Ja/Nej/Delvis] Programmet är kompilerbart och körbart på en dator under både Mac, Linux och MS Windows (alltså inga plattformspecifika konstruktioner) med SDL och SDL_ttf, SDL_image.
Kommentar: Ja

Programmet använder endast SDL-bibliotek och standard C++ och innehåller inga plattformspecifika konstruktioner. Programmet kan därför kompileras och köras på MacOS, Linux och Windows förutsatt att SDL3, SDL3_image och SDL3_ttf finns installerade.

4. Krav på den Specifika Delen(Spelet som använder sig av Spelmotorn)

4.1. [Ja/Nej/Delvis] Spelet simulerar en värld som innehåller olika typer av visuella objekt. Objekten har olika beteenden och rör sig i världen och agerar på olika sätt när de möter andra objekt.
Kommentar: Ja

Spelet innehåller flera typer av visuella objekt såsom spelaren, fiender och projektiler. Dessa objekt har olika beteenden och interagerar med varandra i spelvärlden.

4.2. [Ja/Nej/Delvis] Det finns minst två olika typer av objekt, och det finns flera instanser av minst ett av dessa objekt.
Kommentar: Ja

Spelet innehåller flera olika typer av objekt:

Player
Enemy
Bullet

Det finns flera instanser av fiender samtidigt i spelet

4.3. [Ja/Nej/Delvis] Figurerna kan röra sig över skärmen.
Kommentar: Ja

Figurerna kan röra sig över skärmen. Spelaren kan röra sig i alla riktningar och fiender rör sig nedåt över skärmen.

4.4. [Ja/Nej/Delvis] Världen (spelplanen) är tillräckligt stor för att den som spelar skall uppleva att figurerna förflyttar sig i världen.
Kommentar: Ja

Spelvärlden är lika stor som spel-fönstret och figurer rör sig över hela skärmen vilket gör att spelaren tydligt upplever rörelse i världen.

4.5. [Ja/Nej/Delvis] En spelare kan styra en figur, med tangentbordet eller med musen.
Kommentar: Ja

Spelaren styr en figur med tangentbordet. Rörelse sker med tangenterna W, A, S och D och skjutning sker med SPACE.

[Ja/Nej/Delvis] Det händer olika saker när objekten möter varandra, de påverkar varandra på något sätt.
Kommentar: Ja

När objekt möter varandra påverkar de varandra. Om en projektil kolliderar med en fiende tas båda objekten bort från spelet. Om en fiende når botten av skärmen avslutas spelet och texten "GAME OVER" visas.